

Exercise Sheet 5

Efficient Computing/Parallel Computing 2 in SS 2026

Date of issue: 5.5.2026 Deadline: 12.5.2026

The exercises marked with ★ are relevant for admission to the examination and must be submitted in electronic form in Moodle by the submission deadline.

Exercise 5.1 (*Estimating cache line size by strided pointer chasing*).

Write a C program to investigate the impact on performance of cache misses and estimate the cache line size of a given hardware architecture. Therefore, implement a method that performs strided pointer chasing on a vector $x = (x_0, x_1, \dots, x_{n-1})$ of size $n \in \mathbb{N}$. Namely, instead of directly accessing array elements, the vector should be initialized such that each entry of the vector stores the index j_i of the next element to visit. The indices follow the recursion

$$j_i := (j_{i-1} + k) \bmod n \quad \text{for } i = 1, 2, \dots$$

with initial value $j_0 = 0$, where the parameter $k \in \mathbb{N}$ is called the stride.

- Allocate a vector x of size n and suitable data type.
- Initialize x such that each entry stores the index of the next element with stride k .
- Implement a function for (strided) pointer chasing, namely, a function that performs $m \in \mathbb{N}$ memory accesses by following the indices in x starting from with the pointer x_0 .
- Avoid compiler optimizations by using volatile variables and the compiler flag `-O0`.
- Minimize cache warm-up effects by clearing the cache before runtime measurements, e.g., access a sufficiently large auxiliary array to override stored information in the cache.
- Measure the runtime of the pointer chasing for varying parameters n , k , and m and compute the mean/median time per access.
- Visualize the average runtime results as functions of the parameters.
- Analyze the results w.r.t. cache properties such as cache sizes and cache line size.
- (Optional) Investigate the relation of the measurements with the specification of the used hardware, e.g., a Raspberry Pi (`pirate01`, `pirate02`, ...) in the lab room.

Exercise 5.2 (*Matrix-vector products in the ideal-cache model*).

Consider a two-dimensional array A with m rows and n columns and a vector x with n rows. Suppose that there is an implementation of the matrix-vector product $y \leftarrow Ax$ of the form

```
for(int i=0; i<m; i++){
    y[i] = 0;
    for(int j=0; j<n; j++){
        y[i] = y[i] + A[i][j]*x[j];
    }
}
```

that returns the result in a vector $\mathbf{y}$ with m rows. Assume that there is another implementation of the transposed matrix-vector product $\mathbf{y} \leftarrow A^T \mathbf{x}$ that uses the same data structure for the array A and a similar nesting of loops, but whose innermost loop body accesses $A[j][i]$ rather than $A[i][j]$.

```
for(int i=0; i<n; i++){
    y[i] = 0;
    for(int j=0; j<m; j++){
        y[i] = y[i] + A[j][i]*x[j];
    }
}
```

Note that this algorithm requires a suitable change of the sizes of the vectors $\mathbf{x}$ and $\mathbf{y}$. Both algorithms are implemented in the language C.

Analyze the two different algorithms in the ideal-cache model with cache size M and cache-line size B . For the sake of simplicity, suppose that the size of the matrix and vector elements is one word.

- a) Derive the cache complexity of the two algorithms for the case $mn + m + n \ll M$.
- b) Derive the cache complexity of the two algorithms for the case $mn + m + n \gg M$ and $\min(m, n) \gg M/2$.
- c) Implement the row- and column-based variants on an architecture of your choice. Find out the values of M and B for your architecture. Measure the running times of your implementations and compare them with the predictions based on the ideal-cache model.